算法基础：习题解答 3

2026 春　任课老师：陈雪　　due: April 15, 23:59

作业要求：说明思路与符号，清晰简洁的伪代码，必要的时间复杂度分析和必要的正确性分析。可以直接调用基本的数据结构和已讨论过的算法 / 程序（如排序、找中位数、二分查找等）。

问题 1

Solution.

(a) 根据题意，有 $a = 3, b = 2$，那么 $\log_b a = \log_2 3$。而

$$f(n) = O(n) = O\big(n^{\log_2 3-(\log_2 3-1)}\big) = O\big(n^{\log_2 3-\Omega(1)}\big).$$

根据主定理，有 $T(n) = \Theta(n^{\log_2 3})$。

(b) 根据题意，有 $a = 9, b = 3$，那么 $\log_b a = 2$。而

$$f(n) = n^2 = \Theta(n^2).$$

根据主定理，有 $T(n) = \Theta(n^2 \log n)$。

(c) 考虑递归树，共有 $\log_2 n$ 层。第 i 层有 2^i 个节点，每个节点需要解决的子问题大小是 $n/2^i$。第 i 层的运算量为

$$2^i \frac{n/2^i}{\log_2(n/2^i)} = \frac{n}{\log_2 n - i}.$$

对所有层数累加计算总和，有

$$\sum_{i=0}^{\log_2 n-1} \frac{n}{\log_2 n - i} = \sum_{i=1}^{\log_2 n} \frac{n}{i} = \Theta(n \log\log n).$$

叶子层的总开销为 $\Theta(n)$，因此 $T(n) = \Theta(n \log\log n)$。

问题 2

Solution.

Algorithm 1 Fast Hadamard Transform

```
1: procedure FHT(n, x[0], x[1], ..., x[2^n − 1])
2:     if n = 0 then
3:         return [x[0]]
4:     end if
5:     S_0 ← FHT(n − 1, x[0], x[1], ..., x[2^{n−1} − 1])
6:     S_1 ← FHT(n − 1, x[2^{n−1}], x[2^{n−1} + 1], ..., x[2^n − 1])
7:     for i = 0, 1, ..., 2^{n−1} − 1 do
8:         y[i] ← S_0[i] + S_1[i]
9:         y[2^{n−1} + i] ← S_0[i] − S_1[i]
10:    end for
11:    return y
12: end procedure
```

根据定义，有（向量的最后一位对应数组的高位）

$$\begin{aligned}
\widehat{x}(v) &= \sum_{u\in\{0,1\}^n} (-1)^{\langle v,u\rangle} x(u) \\
&= \sum_{u\in\{0,1\}^{n-1}\times\{0\}} (-1)^{\langle v,u\rangle} x(u) + \sum_{u\in\{0,1\}^{n-1}\times\{1\}} (-1)^{\langle v,u\rangle} x(u) \\
&= \sum_{u\in\{0,1\}^{n-1}\times\{0\}} (-1)^{\langle v_{[1,n-1]},u_{[1,n-1]}\rangle} x(u) \\
&\quad + \sum_{u\in\{0,1\}^{n-1}\times\{1\}} (-1)^{\langle v_{[1,n-1]},u_{[1,n-1]}\rangle + v_n} x(u) \\
&= \sum_{u\in\{0,1\}^{n-1}} (-1)^{\langle v_{[1,n-1]},u\rangle} x(u,0) + (-1)^{v_n} \sum_{u\in\{0,1\}^{n-1}} (-1)^{\langle v_{[1,n-1]},u\rangle} x(u,1).
\end{aligned}$$

令 $v' = v_{[1,n-1]}$，然后设

$$\widehat{x}'(v') = \sum_{u\in\{0,1\}^{n-1}} (-1)^{\langle v',u\rangle} x(u,0), \qquad \widehat{x}''(v') = \sum_{u\in\{0,1\}^{n-1}} (-1)^{\langle v',u\rangle} x(u,1).$$

那么前者是 $n-1, x_0, x_1, \ldots, x_{2^{n-1}-1}$ 的子问题，后者是 $n-1, x_{2^{n-1}}, x_{2^{n-1}+1}, \ldots, x_{2^n-1}$ 的子问题。

接下来，将以上结果带入原式：

$$\widehat{x}(v) = \widehat{x}'(v') + (-1)^{v_n} \widehat{x}''(v').$$

将 v' 相同的部分合并处理，就有：当 $v_n = 0$ 时，

$$\widehat{x}(v', 0) = \widehat{x}'(v') + \widehat{x}''(v');$$

当 $v_n = 1$ 时，

$$\widehat{x}(v', 1) = \widehat{x}'(v') - \widehat{x}''(v').$$

最后考虑边界情况：当 $n = 0$ 时，$\widehat{x}(0) = x(0)$。该过程完全说明了伪代码的功能，从而伪代码是正确的。

根据伪代码的功能，有算法时间复杂度的递推：

$$T(n) = 2T(n-1) + \Theta(2^n).$$

令 $N = 2^n$，并设 $T'(N) = T(\log_2 N) = T(n)$，那么有

$$T'(N) = 2T'(N/2) + \Theta(N).$$

根据主定理，有 $T'(N) = \Theta(N \log N)$，即

$$T(n) = \Theta(n2^n).$$

问题 3

Solution.

Algorithm 2 Eagle-Egg Problem

```
1:  procedure SOLVE(n, k)
2:      if n = 0 then
3:          return 0
4:      end if
5:      k ← min{k, ⌈log_2(n + 1)⌉}
6:      if k = 1 then
7:          return n
8:      end if
9:      for i = 0, 1, ..., k do
10:         f_{i,0} ← 1
11:     end for
12:     for j = 1, 2, ..., 2⌈√n⌉ do
13:         f_{0,j} ← 1
14:         for i = 1, 2, ..., k do
15:             f_{i,j} ← f_{i−1,j−1} + f_{i,j−1}
16:         end for
17:         if f_{k,j} ≥ n + 1 then
18:             return j
19:         end if
20:     end for
21: end procedure
```

引理 1. *若 $k \geq \lceil \log_2(n+1) \rceil$，则最坏情况下所需的最少试验次数恰为 $\lceil \log_2(n+1) \rceil$。*

证明. 耐久值（答案）是一个 $[0, n]$ 之间的整数，共有 $n+1$ 种可能。

先证上界。若有至少 $\lceil \log_2(n+1) \rceil$ 个鹰蛋，则可以始终在当前区间的中点试验。每次试验后，若鹰蛋碎，则答案落在较低的一半区间；若未碎，则答案落在较高的一半区间。于是经过一次试验后，候选状态数至多为原来的二分之一向上取整。故经过 $\lceil \log_2(n+1) \rceil$ 次试验后，一定可以将 $n+1$ 个可能状态缩小到 1 个，从而确定耐久值。

再证下界。任意一个进行 m 次试验的算法，其每次试验至多只有“碎 / 不碎”两种结果，因此整棵决策树至多有 2^m 个叶子，也就至多能区分 2^m 种状态。为了区分全部 $n+1$ 种耐久值，必须有

$$2^m \geq n+1, \qquad m \geq \lceil \log_2(n+1) \rceil .$$

综上，最少试验次数恰为 $\lceil \log_2(n+1) \rceil$。故当 $k \geq \lceil \log_2(n+1) \rceil$ 时，更多的鹰蛋不会进一步减少最少试验次数。 □

引理 2. *若 $k \geq 2$，那么试验次数不超过 $2\lceil \sqrt{n} \rceil$。*

证明. 给出构造证明。先用第一个鹰蛋试验 $\lceil \sqrt{n} \rceil, 2\lceil \sqrt{n} \rceil, \ldots$，以 $\lceil \sqrt{n} \rceil$ 的间隔从小到大依次测试（最后一次至多测试到 n），最多使用 $\lceil \sqrt{n} \rceil$ 次使得将耐久度确定在一个长度不超过 $\lceil \sqrt{n} \rceil$ 的区间上。

再使用第二个鹰蛋在该区间上从小到大依次尝试，使用 $\lceil \sqrt{n} \rceil - 1$ 次试验就可以确定耐久度的大小。总次数不超过 $2\lceil \sqrt{n} \rceil$。 □

根据引理 1，不妨设 $2 \leq k \leq \lceil \log_2(n+1) \rceil$。设 $f_{i,j}$ 表示用 i 个鹰蛋，使用 j 次试验可以区分多少种状态。

边界条件：

1. 如果没有鹰蛋，只能确定至多一种状态；
2. 如果试验次数为 0，只能确定至多一种状态。

即 $f_{i,0} = f_{0,j} = 1$。

转移方程：考虑有 i 个鹰蛋试验 j 次最多可以区分多少种状态。如果鹰蛋被摔碎，则变为子问题 $i-1$ 个鹰蛋试验 $j-1$ 次；否则变为子问题 i 个鹰蛋试验 $j-1$ 次。由于进行试验后有两种互不相交的分支，因此

$$f_{i,j} = f_{i-1,j-1} + f_{i,j-1}.$$

获取答案：从小到大枚举试验次数 j，确定最小的 j 使得 $f_{k,j} \geq n+1$。根据引理 2，必然有 $f_{k,2\lceil\sqrt{n}\rceil} \geq n+1$，故不需要计算 $j > 2\lceil\sqrt{n}\rceil$ 的部分。

由于上述定义完整刻画了“一次试验后按碎 / 不碎分治”所能区分的全部状态数，因此该动态规划正确计算了最少的最坏情况下的试验次数。

当 $k = 1$ 时，答案为 n；当 $k > 1$ 时，设答案为 j^*，时间复杂度为 $\Theta(kj^*) \subseteq O(\sqrt{n}\log n) = o(n)$。

另外思路：还可以用下面的等式证明引理 1 和引理 2：

$$f_{k,m} = 1 + \sum_{i=1}^{k} \binom{m}{i} \qquad (\binom{m}{i} = 0 \text{ 当} i > m),$$

问题 4

Solution.

Algorithm 3 LIS

```
1: procedure LIS(n, a_1, a_2, ..., a_n)
2:     b_0 ← −∞; b_i ← +∞, ∀i ∈ [1, n+1]
3:     a_0 ← −∞, a_{n+1} ← +∞; f_0 ← 0
4:     for i = 1, 2, ..., n+1 do
5:         x ← BinarySearch(b_{[0,n]}, a_i)
       // 返回 b_0, ..., b_n 中严格小于 a_i 的元素个数。
6:         b_x ← min{b_x, a_i}; f_i ← x
7:     end for
8:     x ← n+1
9:     for i = n, n−1, ..., 1 do
10:        if f_i = f_x − 1 and a_i < a_x then
11:            ans_{f_i} ← a_i; x ← i
12:        end if
13:    end for
14:    return ans
15: end procedure
```

设 f_i 表示结尾为 i 的最长上升子序列的长度。显然 $f_i \leq i$，特别地，$f_0 = 0$。

引理 3.

$$f_i = \max_{0\leq j<i,\ a_j<a_i} \{f_j + 1\}.$$

证明. 先证 $f_i \geq \max_{0\leq j<i,\ a_j<a_i}\{f_j+1\}$。对于任意满足 $0 \leq j < i, a_j < a_i$ 的 j，将一个以 j 结尾、长度为 f_j 的上升子序列接上 a_i，可得到一个以 i 结尾的上升子序列，长度为 $f_j + 1$。因此对所有这样的 j 取最大值，仍有上述不等式。

再证 $f_i \leq \max_{0\leq j<i,\ a_j<a_i}\{f_j+1\}$。设以 i 为结尾的最长上升子序列为 $a_{i_1}, a_{i_2}, \ldots, a_{i_{f_i}}$，其中 $1 \leq i_1 < i_2 < \cdots < i_{f_i} = i$。若 $f_i = 1$，则显然成立。以下考虑 $f_i > 1$，必有 $i_{f_i-1} < i$ 且 $a_{i_{f_i-1}} < a_i$，并且以 i_{f_i-1} 为结尾的最长上升子序列长度为 $f_i - 1$，即 $f_{i_{f_i-1}} = f_i - 1$。因此

$$\max_{0\leq j<i,\ a_j<a_i} \{f_j + 1\} \geq f_i.$$

综上，所述等式成立。 □

实际上，只需要这个性质，就可以使用平衡树解决 f_i 的递推。动态维护 (a_j, f_j)，$j < i$ 的信息时，查询需要的信息是：满足 $a_j < a_i$，f_j 最大为多少？

这里继续简化思路。使用以上性质，有

$$\begin{aligned} f_i &= \max_{0\le j<i,\ a_j<a_i} \{f_j + 1\} \\ &= \max_{k\in[0,n]}\ \max_{0\le j<i,\ a_j<a_i,\ f_j=k} \{f_j + 1\} \\ &= \max_{\substack{k\in[0,n],\ \exists j\in[0,i) \\ a_j<a_i,\ f_j=k}} \{k + 1\}. \end{aligned}$$

再设一个辅助数组 b_k 表示：循环过程为 i 时，满足 $f_j = k$ 且 $j < i$ 时，a_j 的最小值。形式化地：

$$b_k = \min_{0\le j<i,\ f_j=k} a_j.$$

若不存在 j 满足 $0 \le j < i, f_j = k$，那么 $b_k = +\infty$。于是

$$f_i = \max_{k\in[0,n],\ b_k<a_i} \{k + 1\}.$$

引理 4. *对于所有非负整数 k，有 $b_k \le b_{k+1}$ 成立。*

证明. 用反证法。若 $b_{k+1} = +\infty$，则显然有 $b_k \le b_{k+1}$。以下只需考虑 $b_{k+1} < +\infty$ 的情况。若 $b_k > b_{k+1}$，设已经处理的某个下标 ℓ 满足 $f_\ell = k + 1$ 且 $a_\ell = b_{k+1}$。那么存在 $j \in [0, \ell)$ 使得 $f_j = f_\ell - 1$ 且 $a_j < a_\ell$，有

$$f_j = k, \qquad a_j < a_\ell = b_{k+1} < b_k.$$

因此 $b_k \le a_j$，与条件矛盾。 □

根据以上性质，$b_k \le b_{k+1}$，那么 b 是一个单调不降序列。从而仅需要找到满足 $b_k < a_i$ 的最大的 k，记为 k^*。这个过程可以进行二分，返回 $b_0 \sim b_n$ 中有多少元素小于 a_i。设数量为 x，那么 $k^* = x - 1$。

获取 k^* 后，由于 $x = k^* + 1$，令 $f_i = x$，并更新 $b_x \leftarrow \min\{b_x, a_i\}$。

构造答案：根据引理 3，对任意 $i \in [1, n + 1]$，都存在某个 $j \in [0, i)$ 使得 $f_j = f_i - 1$ 且 $a_j < a_i$。定义 $a_{n+1} = +\infty$，则新问题上的最长上升子序列必然以 $n + 1$ 结尾，且其长度为原问题的最长上升子序列长度加 1。

因此可以从 $x = n + 1$ 开始倒序扫描，依次寻找满足 $f_i = f_x - 1$ 且 $a_i < a_x$ 的前驱 i，并将 a_i 作为答案中的一个元素加入，即令 $ans_{f_i} \leftarrow a_i$，随后令 $x \leftarrow i$ 继续回溯，即可恢复出一个最长上升子序列。

时间复杂度：

- 计算答案长度：循环 $O(n)$ 次，每次循环内使用了 $O(\log n)$ 的运算量，总时间复杂度为 $\Theta(n \log n)$。
- 构造答案：伪代码循环 $O(n)$ 次，每次循环内运算量为 $\Theta(1)$，时间复杂度为 $\Theta(n)$。

总时间复杂度为 $\Theta(n \log n)$。